

Guide d'exploitation — Entrepôt de Données de Santé

Module Big Data M2 · Épreuve E05

L'automatisation du pipeline, la gestion des erreurs, la journalisation, la traçabilité — et la conduite au quotidien : lancer, rejouer, reprendre après un incident, maintenir.

Les choix de conception et leur justification sont dans le [Rapport de conception](#).

Sommaire

1. Ce qui déclenche le pipeline
2. Ce qui se passe quand ça échoue
3. La journalisation
4. La traçabilité, de bout en bout
5. Utilisation et maintenance
6. Les évolutions depuis la première analyse

La Partie 2 du sujet demande deux choses distinctes : que la collecte et la transformation soient **planifiées**, avec gestion des erreurs, journalisation et traçabilité ; et qu'une **documentation d'utilisation et de maintenance** existe — lancement, reprise sur incident. Ce chapitre couvre les deux, et retrace en clôture ce qui a changé depuis la première analyse (§ 6). Il s'ajoute aux parties 1 à 3 sans les modifier : leurs choix restent ceux qu'elles décrivent.

1. Ce qui déclenche le pipeline

```
10 3 * * * cd $EDS_HOME && .venv/bin/python -m eds.run >> logs/cron.log 2>&1
```

03h10, et pas un autre horaire. Le choix est conventionnel : après une nuit de dépôt supposée terminée, avant la prise de poste du matin — de sorte qu'un incident se découvre en arrivant, jamais en pleine journée de soin. Il n'est pas ajusté sur une fenêtre de dépôt observée : le dépôt fourni date chaque jour au jour près (`_jour_depot` , type `Date` en bronze — rapport, § 3.2), sans horodatage intra-journalier, et les fichiers du dépôt courant portent tous la même date de fichier, preuve d'une extraction unique plutôt que de dépôts échelonnés dans la nuit. Rien dans ce dépôt ne permet donc de mesurer une fenêtre réelle ; 03h10 reste une hypothèse à confirmer en production, pas une valeur calée sur une observation. L'heure est un paramètre du fichier `ops/crontab.example` , pas une constante du code : la déplacer ne touche à rien d'autre.

cron , et pas un ordonnanceur applicatif. Airflow, Dagster ou un service managé apportent un graphe de dépendances entre tâches, des reprises automatiques et une interface de supervision. Aucun des trois n'a d'usage ici : le pipeline est **une seule tâche**, appelée une fois par jour, dont l'orchestration interne (`schema` → `lake` → `bronze` , par jour → `referentiels` → `silver` → `gold` → `droits` — sept des huit noms d'étape du § 3, le huitième, `restitution` , relevant d' `eds.restitution`) est déjà écrite en Python dans `eds/run.py` — le graphe de dépendances existe, mais à l'intérieur d'un seul processus, pas entre plusieurs tâches cron. Ajouter un ordonnanceur ne changerait aucune de ces dépendances ; il ajouterait une base de métadonnées et un serveur web à faire tourner pour planifier un appel quotidien. C'est un choix à revoir le jour où plusieurs pipelines indépendants doivent se coordonner — pas avant.

Ce que **cron** ne garantit pas, et ce qu'on met en face.

cron ne fait pas	Ce qui compense
Reprise automatique après un échec	Rien ne relance seul une exécution en échec — c'est assumé (§ 5) : cron retente le lendemain à 03h10, jamais dans l'heure. En attendant, l'échec est explicite (code de sortie non nul) et entièrement tracé (§ 3), pour qu'un humain corrige et relance
Alerte en cas d'échec	Aucune n'est câblée aujourd'hui : la sortie <code>logs/cron.log</code> et <code>ops.executions</code> sont passives , il faut aller les lire. C'est une limite assumée à ce stade (recommandation rapport, § 3.4 — brancher un outil de supervision reste à faire)
Dépendance entre tâches	Sans objet ici : une seule ligne crontab, une seule commande. Le graphe interne (<code>Pipeline.executer</code>) reste dans le processus Python, pas dans cron

Le mode par défaut est incrémental, et c'est un choix. `eds.run` sans option calcule `jours_disponibles()` – `jours_deja_ingeres()` : seuls les jours absents de l'entrepôt sont chargés. Un rechargement complet (`--tout`) recopierait chaque nuit 28 fichiers pour n'en traiter réellement qu'un — au prix, à ce volume, de rester instantané, mais le principe ne tient pas au-delà de quelques dizaines de milliers de lignes par jour (rapport, § 3.3, recalcul intégral de silver et gold). L'incrémental borne le travail nocturne à ce qui a réellement changé.

Ce qui se passe si un jour manque. `jours_disponibles()` est l'union des répertoires réellement présents sous `source-filestorage/` — pas un calendrier attendu. Si le CHU ne dépose rien une nuit, ce jour n'existe simplement pas dans la liste : l'exécution suivante n'échoue pas, elle **n'a rien à faire** pour ce jour, et se termine normalement. Le jour sera repris automatiquement dès qu'il apparaîtra dans le dépôt, sans action manuelle — mais rien n'alerte aujourd'hui si le CHU cesse durablement de déposer : c'est la même limite que la ligne « pas d'alerte » ci-dessus, appliquée à l'absence de dépôt plutôt qu'à l'échec d'un traitement.

2. Ce qui se passe quand ça échoue

Deux familles d'erreurs, parce qu'elles n'appellent pas la même réponse.

```
class ErreurPipeline(Exception):
    """Échec métier : la relance à l'identique ne changera rien."""

    def _est_transitoire(erreur: Exception) -> bool:
        return isinstance(erreur, OperationalError) or (
            isinstance(erreur, DatabaseError) and "Connection" in str(erreur)
        )
```

Une **panne transitoire** — le moteur ClickHouse qui redémarre, une connexion coupée — a de bonnes chances de disparaître d'elle-même en quelques secondes : elle est retentée. Une **erreur métier** — `ErreurPipeline`, ou toute erreur qui n'est ni une `OperationalError` ni une `DatabaseError` évoquant une connexion — ne l'est pas : un jour absent du dépôt, un SQL invalide, une partition déjà verrouillée resteront faux à la deuxième tentative comme à la première. La retenter ne ferait que perdre du temps et bruyter le journal d'un même échec répété trois fois.

`avec_reprises` porte la première famille : **3 tentatives**, avec **temporisation exponentielle** — 2 s avant la deuxième, 4 s avant la troisième (`ATTENTE_INITIALE_S = 2`, doublée à chaque échec). Les trois étapes qui appellent du SQL transformant (`bronze`, `silver`, `gold`) et le chargement du schéma en sont enveloppées ; la lecture du lake, elle, échoue en `ErreurPipeline` sans retenter, puisqu'un fichier absent le reste.

Deux preuves réelles, tirées de `ops.executions`, plutôt qu'un cas fabriqué. La classification n'est pas seulement écrite dans le code, elle s'observe dans l'historique des exécutions déjà journalisées :

Étape	Message (tronqué)	Famille	Comportement observé
lake	ErreurPipeline: aucun fichier trouvé pour le 2099-01-01	métier	échec immédiat, aucune retentative — répété à chaque jeu de <code>tests.demontrer reprise</code> , qui force un jour inexistant
silver	DatabaseError: ... Function with name `quote` does not exist	définitive (ni <code>OperationalError</code> , ni « Connection »)	échec immédiat malgré la classe <code>DatabaseError</code> — une erreur SQL réelle rencontrée en développement, pas retentée puisqu'aucune reprise n'aurait changé une fonction inexistante

Ce que ce tableau ne peut pas montrer, c'est un compte figé : la première ligne se réenrichit d'une occurrence à chaque exécution de la démonstration citée, et `ops.executions` est un journal qui ne s'efface jamais (§ 3). Au moment de la rédaction, la répartition par étape s'obtenait par :

```
SELECT etape, count() AS echecs
FROM ops.executions WHERE statut = 'echec'
GROUP BY etape ORDER BY echecs DESC;
```

```
lake      72
silver    2
gold      1
```

soit 75 échecs sur 412 exécutions distinctes (`run_id`) — un chiffre voué à grossir encore côté `lake` à chaque nouvelle relecture de `tests.demontrer reprise` , sans que cela change rien à la classification qu'il illustre.

La propriété qui rend la reprise triviale : il n'y a rien à restaurer. Bronze est la source de vérité durable — chaque partition (`_jour_depot`) est réécrite en entier (`DROP PARTITION` puis rechargement), jamais modifiée en place. Silver et gold sont **recalculés intégralement** à chaque passage (`TRUNCATE` puis `INSERT` — rapport, § 2.7). Le schéma, lui, ne détruit jamais : les six fichiers de `SCHEMA` sont tous en `CREATE TABLE IF NOT EXISTS` , rejouables sans effet de bord. La conséquence directe : après un échec, **corriger la cause et relancer suffit**. Aucune sauvegarde à restaurer, aucun script de rattrapage distinct de `eds.run` lui-même.

Ce que le pipeline garantit après un échec : un entrepôt cohérent, jamais à moitié écrit. `tests.demontrer reprise` le vérifie, pas seulement l'affirme. Rejoué à l'instant :

- | | |
|-------------------------------|---|
| ① Jour de dépôt malformé | → code de sortie 1, rejeté avant écriture |
| ② Jour absent du dépôt du CHU | → code de sortie 1, échec explicite |
| ③ Traçabilité de l'échec | → présent dans <code>ops.executions</code> |
| ④ Cohérence après incident | → inchangé : { <code>bronze.sejours: 6797</code> , <code>silver.sejours: 6729</code> , <code>gold_pilotage.fact_sejour: 6729</code> } |
| ⑤ Reprise par simple relance | → code de sortie 0, entrepôt rétabli à l'identique |
| ⑥ Chargement partiel | → <code>bronze.monitoring</code> du 2026-08-27 : 321 → 0 → 321 lignes |

Le point ⑥ est le cas qui compte réellement : une partition entière de `bronze.monitoring` est supprimée pour simuler un jour à moitié chargé — un scénario où une source aurait échoué après une autre (§ 1, `jours_deja_ingeres` , qui exige que **toutes** les sources d'un jour soient présentes, pas seulement `sejours`). Une simple relance de `eds.run` retrouve les 321 lignes disparues, sans argument spécial ni intervention : le jour est redétecté comme non entièrement ingéré, et rechargé.

3. La journalisation

Deux destinations, parce qu'elles servent deux usages qu'un seul support ne couvre pas. `logs/pipeline.log` est un fichier — une ligne JSON par événement, lisible par `tail -f`, par un humain en urgence à 3h du matin comme par un outil de collecte de logs (`FormateurJSON`, `eds/journal.py`). Il survit même si ClickHouse est indisponible, puisqu'il n'en dépend pas. `ops.executions` est une **table** — interrogeable en SQL, jointe aux données qu'elle décrit par `_run_id`, agrégeable (compter les échecs par étape, mesurer une durée moyenne). Le fichier répond à « que s'est-il passé, dans l'ordre ? » ; la table répond à « quelle exécution a produit cette ligne, et avec quel bilan ? ». Un seul des deux ne remplirait pas l'autre rôle : un fichier ne se `JOIN`-e pas à `bronze.sejours` sur `_run_id`, et une table disparaît si le moteur qui la porte est lui-même la panne à diagnostiquer.

Structure de `ops.executions` :

Colonne	Contenu
<code>run_id</code>	identifiant de l'exécution, partagé avec <code>_run_id</code> dans <code>bronze/silver/gold</code>
<code>etape</code>	<code>schema</code> , <code>lake</code> , <code>bronze</code> , <code>referentiels</code> , <code>silver</code> , <code>gold</code> , <code>droits</code> , <code>restitution</code>
<code>jour</code>	jour de dépôt concerné — <code>NULL</code> pour les étapes non journalières
<code>statut</code>	<code>succes</code> ou <code>echec</code>
<code>lignes</code>	volume traité par l'étape
<code>duree_s</code>	durée mesurée
<code>message</code>	vide en succès ; type et texte de l'exception en échec, tronqué à 500 caractères
<code>demarre_a</code> , <code>termine_a</code>	horodatages

Une requête d'exploitation utile — les dernières exécutions, succès comme échecs, celle que `docs/RAPPORT.md` et `README.md` recommandent en premier réflexe :

```
SELECT demarre_a, run_id, etape, statut, lignes, duree_s, message
FROM ops.executions ORDER BY demarre_a DESC LIMIT 20;
```

Les chiffres réels du journal, au moment de la rédaction : `logs/pipeline.log` compte 42 743 lignes ; `ops.executions` porte 412 exécutions distinctes (`run_id`), pour 8 étapes possibles et 75 étapes en échec (§ 2) — le reste en succès. Ces trois chiffres ne sont pas figés : chaque relance de `eds.run` ou de `tests.demontrer` ajoute ses propres lignes et sa propre exécution, sans jamais rien effacer — c'est un journal, pas un instantané. Le fichier de bord accumule une ligne par étape et par exécution (accessoirement plus dense que la table pendant le développement, où chaque relance de test ajoute ses propres lignes) ; les deux destinations restent cohérentes entre elles, puisqu'écrites par le même code au même instant (`Pipeline.etape`, § 2).

Ce qui n'y entre jamais : aucune donnée de santé, aucun pseudonyme, aucun mot de passe ni jeton. Les messages ne portent que des métadonnées d'exécution — nom d'étape, jour, compte de lignes, type d'exception. Ce n'est pas seulement une promesse de l'en-tête de `eds/journal.py` : `tests.verifier rgpd` le vérifie automatiquement, à chaque passage, en balayant le fichier de log et la colonne `message` d'`ops.executions` avec un motif qui repère un pseudonyme (16 caractères hexadécimaux), un IPP (`IPP` suivi de 7 chiffres) ou un NIR (15 chiffres) :

```
OK    logs/pipeline.log sans identifiant patient
OK    ops.executions sans identifiant patient
```

Le point faible, assumé plutôt que caché. `_consigner` — la fonction qui écrit dans `ops.executions` — est elle-même enveloppée dans un `try/except` :

```
except Exception:
    # Ne jamais faire échouer le pipeline à cause de son propre
    # journal : le fichier de log reste la trace de secours.
    LOG.warning("journal ClickHouse indisponible", exc_info=True)
```

Le choix est délibéré — un journal qui peut faire échouer ce qu'il journalise serait pire que l'absence de journal — mais il a un coût : si ClickHouse tombe **au moment précis** où une étape se termine, l'écriture dans `ops.executions` échoue silencieusement (avertissement seul), et `logs/pipeline.log` devient la **seule** trace de cette étape. C'est exactement pourquoi les deux destinations coexistent plutôt qu'une seule : le fichier ne dépend d'aucun composant que le pipeline lui-même pourrait avoir mis en défaut.

4. La traçabilité, de bout en bout

Le rapport, § 3.2 annonce la propriété : chaque ligne porte son fichier d'origine et l'exécution qui l'a produite, en une requête, sans jointure entre couches. Voici la démonstration, sur un indicateur réellement affiché — « Passages aux urgences (service) », la série journalière du tableau de bord « Pilotage hospitalier » — remonté jusqu'au fichier de dépôt et à l'exécution qui l'a traité. Chaque étape est **une** requête ; aucune n'en joint deux couches.

① Le tableau de bord affiche, pour le 1er août, 46 passages.

```
SELECT jour, nb_passages_urgences FROM gold_pilotage.kpi_urgences_jour
WHERE jour = '2026-08-01';
```

```
2026-08-01    46
```

② Ce chiffre est un `countIf` sur `fact_sejour` — un séjour parmi les 46, au hasard.

```
SELECT stay_id FROM gold_pilotage.fact_sejour
WHERE date_admission = '2026-08-01' AND service_code = 'URGENCES'
ORDER BY stay_id LIMIT 1;
```

```
S00000445
```

③ Ce séjour, en silver, porte déjà son fichier et son exécution — sans jointure.

```
SELECT patient_pseudo, _fichier_source, _run_id, _built_at
FROM silver.sejours WHERE stay_id = 'S00000445';
```

```
d54c39c15c6fbf94    lake/sejours/2026-08-01/sejours.csv    a444489b6fe0    2026-09-02 21:19:44
```

④ La même ligne, en bronze, avant tout nettoyage — même fichier source, plus l'horodatage d'ingestion.

```
SELECT patient_pseudo, _fichier_source, _ingested_at, _run_id
FROM bronze.sejours WHERE stay_id = 'S00000445';
```

```
d54c39c15c6fbf94    lake/sejours/2026-08-01/sejours.csv    2026-09-02 21:08:35    40ae712db09b
```

Le `_run_id` diffère entre ③ et ④ (`a444489b6fe0` contre `40ae712db09b`) — c'est attendu, pas une anomalie. Bronze est incrémental : cette partition n'a plus été touchée depuis son premier chargement, donc son `_run_id` reste celui de ce chargement. Silver, lui, est **recalculé intégralement** à chaque exécution de `eds.run` (rapport, § 2.7) : son `_run_id` / `_built_at` désignent la dernière reconstruction complète de la table, pas la dernière fois que cette ligne précise a changé de valeur. Les deux couches partagent toujours `_fichier_source` — la chaîne de traçabilité tient sur cette colonne, pas sur l'égalité (accidentelle, et non garantie) des `_run_id`. C'est pour la même raison que ces valeurs, rejouées après un nouveau `eds.run`, ne seront plus celles imprimées ci-dessus : elles décrivent un état observé, pas une propriété stable de l'entrepôt.

⑤ Et l'exécution `40ae712db09b`, à l'étape qui a chargé précisément ce fichier, est dans `ops.executions`.

```
SELECT etape, jour, statut, lignes, duree_s
FROM ops.executions WHERE run_id = '40ae712db09b' AND etape = 'bronze' AND jour = '2026-08-01';
```

bronze	2026-08-01	succes	2487	0.018
--------	------------	--------	------	-------

De la carte du tableau de bord au fichier `sejours.csv` déposé le 1er août et à l'exécution qui l'a chargé en 18 millisecondes, la chaîne tient en cinq requêtes, chacune sur une seule table. C'est la réponse concrète à « d'où vient cette donnée, et quand a-t-elle été traitée ? » — le rapport, § 3.2 en énonçait la possibilité ; ce paragraphe l'exécute.

Pourquoi gold s'arrête à `_run_id` et `_built_at`, sans `_fichier_source`. La chaîne ci-dessus le montre en pratique : dès qu'une investigation atteint gold, elle continue avec le compte d'exploitation (rapport, § 2.3), qui lit bronze et silver — c'est là, et seulement là, que vit la colonne fichier. Répéter cette colonne dans gold n'ajouterait rien qu'un doublon, sur une base dont le compte de pilotage n'a de toute façon pas le droit de lire `stay_id`.

5. Utilisation et maintenance

TROIS NIVEAUX DE VÉRIFICATION, ET ILS NE SE REMPLACENT PAS. Les tests unitaires (`pytest`) s'exercent sur les fonctions pures : ils tournent hors ligne, en une fraction de seconde, et disent si une transformation est juste prise isolément. `tests.verifier` s'exécute contre l'entrepôt vivant et confronte chaque indicateur au recalcul depuis les faits : il dit si le système est cohérent avec lui-même. `tests.demontrer` écrit dans l'entrepôt, injecte des lignes fautives puis le remet en état : il dit si les règles **FONCTIONNENT**, y compris celles qu'aucune donnée réelle n'exerce.

Un test unitaire n'atteindra jamais l'équation de conservation sur 87 443 lignes, ni le refus prononcé par le moteur. Une suite qui exige Docker ne tournera jamais en trois secondes dans une chaîne d'intégration. Les trois sont donc gardés.

Commandes d'exploitation courantes :

Besoin	Commande	Effet
Lancement quotidien (celui du cron)	<code>.venv/bin/python -m eds.run</code>	incrémental : ingère les seuls jours absents, reconstruit silver et gold, réapplique les droits
Rejeu d'un jour précis	<code>.venv/bin/python -m eds.run --jour 2026-08-27</code>	réécrit sa partition bronze (<code>DROP PARTITION</code> puis rechargement) ; sans effet sur les autres jours
Rechargement complet	<code>.venv/bin/python -m eds.run --tout</code>	relit les 28 jours ; nécessaire après un changement de jeu de données source, jamais après un simple incident (§ 2)
État de l'entrepôt, sans rien modifier	<code>.venv/bin/python -m eds.run --etat</code>	jours ingérés/en attente, volumes par couche, cinq dernières étapes
Provisionnement de la restitution	<code>.venv/bin/python -m eds.restitution</code>	(re)crée connexions, comptes, droits et tableaux de bord Metabase — idempotent, ~6 s au premier passage, ~1,2 s ensuite
État de Metabase, sans rien modifier	<code>.venv/bin/python -m eds.restitution --etat</code>	connexions et tableaux de bord déjà provisionnés
Tests unitaires, hors ligne	<code>.venv/bin/python -m pytest</code>	85 tests sur les fonctions pures — pseudonymisation, découpage SQL, résolution des référentiels, seuils. Ne demandent ni Docker ni entrepôt , et s'exécutent en moins d'une seconde
Contrôle des 428 propriétés	<code>.venv/bin/python -m tests.verifier</code>	rejoue les cinq sections de vérification (dont <code>conformite</code> , § 6)
Démonstrations rejouables	<code>.venv/bin/python -m tests.demontrer</code>	dont <code>reprise</code> (§ 2) et <code>restitution</code> (cloisonnement vu depuis Metabase)

Deux exécutions mesurées à l'instant, pour donner un ordre de grandeur réel plutôt qu'un chiffre relu ailleurs. Une exécution sans nouveau jour à ingérer (`schema` + `referentiels` + `silver` + `gold` + `droits`) prend **0,24 s** au total sur ce dépôt (0,021 + 0,008 + 0,095 + 0,092 + 0,02, run `e4cd6aeb2e18`) ; la charge initiale des 28 jours prend, elle, environ 1,5 s — c'est ce second chiffre que mesure le démarrage décrit au README. Le provisionnement Metabase suit le même contraste : ~ 6 s la première fois qu'il crée tout, $\sim 1,2$ s ensuite quand il ne fait que réconcilier l'existant.

Reprise sur incident — symptôme, cause, correction. Le tableau ci-dessous reprend celui du README et le complète des cas rencontrés depuis, notamment en développement (colonne « Origine ») :

Symptôme	Cause	Correction	Origine
ClickHouse ne voit pas le lake	Le répertoire lake/ a été supprimé : le montage Docker pointe sur un inode disparu	<code>docker compose restart clickhouse</code>	<code>eds.run</code> , contrôle explicite avant lecture
Connection reset by peer / toute OperationalError	ClickHouse redémarre	Aucune — reprise automatique, temporisation 2 s puis 4 s (§ 2)	classification <code>_est_transitoire</code>
Erreur SQL non liée à une connexion (ex. <code>Function ... does not exist</code>)	Un fichier <code>.sql</code> a été modifié avec une erreur de syntaxe ou une fonction inexistante	Pas de reprise automatique : corriger le SQL, puis relancer	observé en développement (§ 2), 3 occurrences dans <code>ops.executions</code>
Variable d'environnement manquante	<code>.env</code> absent ou incomplet	Copier <code>.env.example</code> en <code>.env</code> et renseigner la valeur manquante — <code>eds.config.exiger</code> nomme précisément la variable en cause dans le message d'erreur	<code>eds.config.exiger</code>
argument invalide	Jour mal formé en ligne de commande	Utiliser le format <code>AAAA-MM-JJ</code>	validation avant toute connexion (<code>valider_jour</code>)
aucun fichier trouvé pour le ...	Jour absent du dépôt du CHU	Vérifier <code>eds-chu-sujet/source-filestorage/</code> ; sans action, le jour sera repris de lui-même dès son dépôt (§ 1)	<code>ErreurPipeline</code> , 72 occurrences dans <code>ops.execution</code> s (dont la démonstration <code>tests.demontrer reprise</code> , rejouable, § 2)
Unknown expression identifier sur une colonne	Un DDL a été modifié : <code>CREATE TABLE IF NOT EXISTS</code> ne migre pas un schéma existant	<code>DROP TABLE ...</code> sur la table concernée, puis relancer le pipeline	§ 2, propriété « rien à restaurer »
Metabase ne répond pas sur <code>/api/health</code> après ...s	Conteneur metabase non démarré, ou JVM encore en cours de démarrage (~1 min la première fois)	<code>docker compose up -d metabase</code> , puis <code>docker compose logs metabase</code> si l'attente échoue malgré tout	<code>eds.restitution</code> , mêmes codes de sortie que <code>eds.run</code> (1 / 2)
connexions / tableaux de bord Metabase absents	<code>eds.restitution</code> n'a jamais été rejoué contre cette instance	<code>.venv/bin/python -m eds.restitution</code>	<code>tests.demontrer restitution</code>
valeurs de référence absentes, section ignorée	<code>eds-chu-sujet/corrigé-kpi-niveau1.json</code> n'est pas sur cette machine (fichier non versionné, § 6)	Aucune correction : la section <code>conformite</code> est volontairement ignorée plutôt que de faire échouer le reste de la suite (§ 6)	<code>tests.verifier conformite</code>
Ports are not available sur Metabase	Le port <code>3000</code> est déjà occupé	Libérer le port (<code>lsof -i :3000</code>) ou republier Metabase sur un autre port	<code>docker-compose.yml</code>

Ce qu'un exploitant fait, dans l'ordre, le matin où le pipeline a échoué pendant la nuit :

1. **Lire `logs/cron.log`** — c'est la sortie brute de la commande `cron`, distincte du journal structuré : elle révèle une panne *système* (Python absent, disque plein, conteneur arrêté) qu' `ops.executions` ne verrait pas, puisque le pipeline n'aurait alors même pas pu s'y écrire.
2. **Interroger `ops.executions`** pour la nuit concernée (la requête du § 3) — identifier l'étape en échec, son message, et si l'exécution a été retentée (§ 2).
3. **Chercher le symptôme dans le tableau ci-dessus** — la cause et la correction sont connues pour les cas déjà rencontrés.
4. **Corriger la cause**, jamais l'effet — pas de tentative de réparer une table à la main : ce serait contredire la propriété du § 2.
5. **Relancer `eds.run`** sans option : idempotent, il retrouve tout seul ce qui manque (jour absent, partition incomplète — § 2, point ⑥).
6. **Vérifier avec `eds.run --etat`**, puis `tests.verifier` — 428 contrôles, dont la conformité aux valeurs de référence si le fichier est présent sur cette machine.
7. **Si l'incident touchait la restitution**, relancer `eds.restitution` — les tableaux de bord ne se resynchronisent pas tout seuls avec un entrepôt corrigé.

Remise à zéro complète (destructif — l'entrepôt et Metabase sont reconstruits intégralement, réservé à un incident qu'aucune correction ciblée ne résout) :

```
docker compose down -v && docker compose up -d
.venv/bin/python -m eds.run --tout
.venv/bin/python -m eds.restitution # -v supprime aussi metabase-data
```

6. Les évolutions depuis la première analyse

Ce que compare cette section. Le dernier commit versionné de ce rapport (`d7a13ab` , « *compléter le § 4 — occupation, mortalité, case-mix et dispersion des durées* ») correspond à l'état remis lors de la première analyse. Tout ce que ce rapport décrivait déjà avant ce chapitre — la décision sur la cohérence temporelle, la section `conformite` , les définitions d'indicateurs actuelles, la restitution Metabase — a été **produit après cette remise**, comme du travail non encore committé (`git diff HEAD --stat` , 19 fichiers, **3 016 lignes ajoutées, 346 retirées** — mesuré en tout dernier lieu, une fois ce chapitre entièrement rédigé, puisque `docs/RAPPORT.md` est lui-même l'un des fichiers du diff et continuerait sinon de fausser sa propre mesure au fil de l'écriture). Ce paragraphe raconte ce que ce diff contient et pourquoi, plutôt que de le laisser dans les seuls messages de commit.

Fichier	Lignes changées	Ce qui y a bougé
<code>tests/demontrer.py</code>	+635	injection de lignes fautives étendue (<code>sejour_coherent</code> , <code>sejour_inconnu</code>), section <code>restitution</code> (cloisonnement vu depuis Metabase)
<code>tests/verifier.py</code>	+502	53 contrôles qualité, 50 contrôles indicateurs, section <code>conformite</code> entière
<code>sql/21_silver_transform.sql</code>	+472	déduplication par <code>row_number()</code> , patient sans identifiant écarté, date de naissance illisible corrigée, <code>releve_hors_sejour</code>
<code>sql/30_gold.sql</code>	+172	quatre vues supplémentaires (occupation, mortalité, case-mix, origine), <code>dms_heures</code> , <code>readmission_30j_brute</code>
<code>README.md</code>	+130	restitution Metabase, quatre comptes ClickHouse, section reprise sur incident
<code>sql/31_gold_transform.sql</code>	+131	recopie directe depuis <code>bronze.sejours</code> plutôt que jointure à <code>silver.sejours</code> (rapport, § 2.8)
<code>docker-compose.yml</code>	+39	service <code>metabase</code> , healthcheck, volume dédié

`docs/RAPPORT.md` (ce chapitre et les évolutions qu'il documente) n'apparaît volontairement pas dans ce tableau : c'est le seul fichier du diff qui grossit pendant qu'on l'écrit, y compris pendant l'écriture de cette phrase — un compte par ligne y serait faux dès la suivante. Il pèse pour l'essentiel de l'écart entre les huit lignes ci-dessus et le total cité plus haut.

Cinq évolutions majeures s'en dégagent.

① **L'alignement sur les valeurs de référence de l'intervenant, et la section `conformite`** . Les fichiers `eds-chu-sujet/corrige-kpi-niveau1.json` et `REPONSES-KPI-niveau1.pdf` , fournis par l'intervenant et marqués « ne pas distribuer », sont désormais dans `.gitignore` au même titre que les données sources — jamais versionnés. `tests.verifier.conformite` les confronte, contrôle par contrôle, aux volumes de silver et aux six indicateurs : effectifs exacts, moyennes à $\pm 0,1$. C'est une section d'une nature différente des quatre autres (rapport, § 2.10) : elle peut échouer sur une **valeur** alors que toutes les **propriétés** tiennent. Absent de cette remise, le fichier de référence ne fait pas échouer la suite : la section s'annonce ignorée et le reste des 428 contrôles s'exécute normalement.

② **La décision sur les séjours temporellement incohérents.** Un séjour dont `discharge_ts < admission_ts` continue d'être écarté de `silver.sejours` — mais ses diagnostics et ses relevés, désormais, ne le suivent plus dans l'exclusion. Vérifié directement sur l'entrepôt : **127** diagnostics et **520** relevés portent aujourd'hui `sejour_coherent = 0` plutôt que d'être en quarantaine.

```
SELECT countIf(sejour_coherent = 0) FROM gold_pilotage.fact_diagnostic; -- 127
SELECT countIf(sejour_coherent = 0) FROM gold_pilotage.fact_releve; -- 520
```

D'où les volumes cités au rapport, § 2.5 et au rapport, § 2.9 : **12 720** diagnostics (12 593 + 127) et **40 920** relevés (40 400 + 520), signalés plutôt que perdus, comme le veut le principe du rapport, § 2.8.

③ **Les définitions d'indicateurs.** Trois redéfinitions, chacune vérifiée à l'instant sur l'entrepôt :

Indicateur	Avant	Après	Valeur vérifiée
Réadmission à 30 jours	seule la définition ajustée (séjours index, décès exclus)	la définition brute devient la référence, l'ajustée reste en complément	780 / 6 729 = 11,59 % brute ; 647 / 5 051 = 12,81 % ajustée
Passages aux urgences	comptés au mode d'admission (<code>admission_mode = 'urgence'</code>)	comptés au service (<code>service_code = 'URGENCES'</code>), le mode reste en complément	1 423 (service) contre 3 327 (mode)
Prévalence par pathologie	filtrée au diagnostic principal seul	tous les types de diagnostic, principal et associé	<code>nb_patients</code> (référence, tous types) contre <code>nb_patients_principal</code> (complément), 11 pathologies dans les deux cas
Seuils d'alerte	valeurs par défaut génériques	FC hors [50, 100] bpm — ceux fixés par l'intervenant	rapport, § 2.10, 3 314 relevés en alerte sur 40 920

④ **Les règles silver ajoutées.** Quatre règles n'existaient pas à la première analyse :

- **Déduplication par version**, et non par `argMax : row_number()` classe la ligne bronze entière par `_jour_depot` décroissant, pour ne pas retomber silencieusement sur une version antérieure quand `birth_year` est `NULL` sur la plus récente (rapport, § 2.9, bug `argMax` démontré empiriquement sur ClickHouse 25.8).
- **Date de naissance illisible corrigée, et non bloquante** : `birth_year` passe à `NULL`, le patient reste en silver — c'est un attribut descriptif, il n'entre dans aucune clé.
- **Patient sans identifiant écarté** : un `patient_id` vide en source (pseudonyme vide après transformation) écarte la ligne, motif `patient_manquant` — pour ne jamais agréger plusieurs individus sous une fausse personne commune.
- **Relevé hors fenêtre du séjour** (`releve_hors_sejour`) : un horodatage de monitoring en dehors de `[admission, sortie]` d'un séjour **cohérent** est écarté — règle dérivée, au-delà de l'énumération littérale du sujet, mais nécessaire pour qu'un relevé reste rattachable à un séjour qui a bien pu le produire.

⑤ **La vue d'origine géographique, et la restitution Metabase.** `kpi_origine_service` (64 lignes, service × département) donne à `region_code` un usage nommé — sans lui, la minimisation RGPD aurait exclu la colonne purement et simplement (rapport, § 3.4). Et l'ensemble de la restitution — deux connexions ClickHouse bornées, quatre comptes Metabase, 22 questions en SQL natif, deux tableaux de bord, provisionnés par API (`eds/restitution.py`, non encore versionné) — a été livré depuis la première analyse : elle n'existait alors qu'à l'état de spécification (§ 1 du sujet), pas d'implémentation.

Ce que ces évolutions apprennent. Les 428 contrôles de `tests.verifier` n'ont, à aucun moment de ce travail, cessé de passer : chaque propriété qu'ils vérifient — équation de conservation, cohérence table $\leftrightarrow$ fait, inclusion numérateur/dénominateur — est restée vraie tout du long, y compris sous l'ancienne définition de la réadmission ou des passages aux urgences. C'est précisément ce qu'une propriété ne peut pas révéler : un dénominateur peut être **cohérent avec lui-même** — se recalculer à l'identique depuis le fait dont il sort, ne jamais dépasser son propre numérateur — sans être **le bon**. Compter les urgences au mode d'admission (3 327) ou au service (1 423) sont deux choix également cohérents ; seule la confrontation aux valeurs de référence de l'intervenant a permis de trancher lequel des deux est attendu. C'est la limite structurelle d'un contrôle de propriété, et la raison d'être de `tests.verifier conformité` : une entreprise ne se contente pas de calculer juste, elle doit calculer la bonne chose.